

GitHub Actions - Lecture 1 Assignment

Overview

This assignment contains 9 questions covering fundamental GitHub Actions concepts including workflows, jobs, Docker integration, environment variables, triggers, and dispatch events.

Prerequisites

Before starting the assignment, you must:

1. **Create a GitHub Repository:**

- Repository name: `github-actions-lecture1-assignment`
- Visibility: Public

2. **Clone the repository to your local machine:**

```
git clone https://github.com/YOUR_USERNAME/github-actions-lecture1-assignment.git
cd github-actions-lecture1-assignment
```

3. **Create the workflows directory:**

```
mkdir -p .github/workflows
```

All workflow files for this assignment should be created in the `.github/workflows/` directory of this repository.

Question 1: Basic Starter Workflow

Objective: Create a workflow that runs on push events and lists the current directory contents.

Requirements:

- Create a workflow file at `.github/workflows/starter.yml`
- Trigger: `push` events
- Single job that:
 - Uses the `actions/checkout` action
 - Prints the directory listing using `ls`

Expected Output: The workflow should check out the repository code and display all files in the working directory.

Question 2: Multi-Shell Hello World

Objective: Create a workflow that prints "Hello World" using three different shells.

Requirements:

- Create a workflow file at `.github/workflows/multi-shell.yml`
- Print "Hello World" using:
 1. **Bash** shell
 2. **PowerShell** shell
 3. **Python** shell

Hint: Use the `shell` keyword in your steps to specify different shell interpreters.

Question 3: Parallel and Sequential Jobs

Objective: Demonstrate job dependencies and execution order.

Requirements:

- Create a workflow file at `.github/workflows/job-dependencies.yml`
- Implement 4 jobs with specific execution patterns:

Parallel Jobs (run simultaneously):

- **job_a:** Print "Running Job A"
- **job_b:** Print "Running Job B"

Sequential Jobs (run in order):

- **job_c:**
 - Must run after `job_a` finishes
 - Print "Running Job C after Job A"
- **job_d:**
 - Must run after `job_c` finishes
 - Print "Running Job D after Job C"

Key Concept: Use the `needs` keyword to create dependencies between jobs.

Question 4: Docker Build and Push

Objective: Automate Docker image building and pushing to Docker Hub.

Requirements:

Setup:

1. Store Docker Hub password in GitHub Secrets: `DOCKER_PASSWORD`
2. Store Docker Hub username in GitHub Variables: `DOCKER_USERNAME`
3. Create workflow at: `.github/workflows/docker-build-push.yml`

Workflow Specifications:

- **Trigger:** Push to `main` branch
- **Actions to Use:**
 - `docker/login-action@v3` for authentication
 - `docker/build-push-action@v6` for building and pushing

Image Tagging:

The workflow must tag images as:

- `docker.io/<username>/myapp:latest`
- `docker.io/<username>/myapp:<commit-sha>`

Example Actions:

```
- name: Log in to Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${vars.DOCKER_USERNAME}
    password: ${secrets.DOCKER_PASSWORD}

- name: Build and push Docker image
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: |
```

```
docker.io/${{ vars.DOCKER_USERNAME }}/myapp:latest
docker.io/${{ vars.DOCKER_USERNAME }}/myapp:${{ github.sha }}
```

Question 5: Environment Variable Scopes & Priority

Objective: Understand environment variable precedence across workflow, job, and step levels.

Requirements:

- Create workflow at: `.github/workflows/env-priority.yml`

Implementation:

1. **Workflow Level:** Define `VAR_EXAMPLE: workflow-level`
2. **Job Level:** Create job `test-env` and override with `VAR_EXAMPLE: job-level`
3. **Steps:**
 - **Step 1:** Print the variable without overriding

```
echo "Step 1 value: $VAR_EXAMPLE"
```

- **Step 2:** Override with `VAR_EXAMPLE: step-level` and print

```
echo "Step 2 value: $VAR_EXAMPLE"
```

Expected Learning Outcome:

Observe the priority order:

- **Lowest Priority:** Workflow-level
- **Medium Priority:** Job-level
- **Highest Priority:** Step-level

Question: Which value appears in Step 1? Which value appears in Step 2?

Question 6: Branch and Path Filtering

Objective: Trigger workflows based on specific branches and file paths.

Requirements:

Setup:

1. Create a new branch: `dev`
2. Create folder structure: `src/`
3. Add file: `src/main.py` with content:

```
print("hello world")
```

Workflow:

- Create workflow at: `.github/workflows/dev-path-trigger.yml`
- **Trigger Conditions:**
 - Push events to `dev` branch only
 - Only when files in `src/` directory are modified

Key Concept: Use `branches` and `paths` filters under the `on.push` trigger.

Question 7: Pull Request with Path Filtering

Objective: Trigger workflows on pull requests with path-specific changes.

Requirements:

Setup:

1. Create branch `stg` from `main`
2. Create folder: `src/`
3. Add a Python file in `src/` directory

Workflow:

- Create workflow at: `.github/workflows/pr-path-trigger.yml`
- **Trigger Conditions:**
 - Pull request events targeting `main` branch
 - Only when files in `src/` directory are changed

Testing:

Create a pull request from `stg` → `main` and verify the workflow triggers only when `src/` files are modified.

Question 8: Workflow Dispatch (Manual Triggers)

Objective: Create a manually triggerable workflow with custom inputs.

Requirements:

- Create workflow at: `.github/workflows/manual-deployment.yml`
- Implement manual trigger with the following inputs:
 - **environment:**
 - Description: 'Deployment environment'
 - Required: true
 - Default: 'staging'
 - **deploy_type:**
 - Description: 'Type of deployment'
 - Required: true
 - Type: choice
 - Options: ['full', 'partial']
 - **dry_run:**
 - Description: 'Run as dry run'
 - Required: false
 - Type: boolean
 - Default: false

Job Requirements:

- Create a job that prints all three input values
- Use `github.event.inputs.<input_name>` to access the values

Task:

1. Create the workflow with `workflow_dispatch` trigger
 2. Navigate to Actions tab in GitHub
 3. Manually trigger the workflow with different input combinations
 4. Observe the output and verify all inputs are correctly displayed
-

Question 9: Repository Dispatch (External Triggers)

Objective: Create a workflow that can be triggered externally via API calls.

Requirements:

- Create workflow at: `.github/workflows/external-trigger.yml`
- Implement repository dispatch event handling
- Listen for event type: `custom-event`

Job Requirements:

- Create a job that:
 - Prints a message indicating it was triggered by an external event
 - Accesses and prints the value from `github.event.client_payload.my_key`

Testing:

Trigger the workflow using GitHub API:

```
curl -X POST \  
  -H "Accept: application/vnd.github+json" \  
  -H "Authorization: Bearer YOUR_GITHUB_TOKEN" \  
  https://api.github.com/repos/OWNER/REPO/dispatches \  
  -d '{"event_type":"custom-event","client_payload":  
{"my_key":"test_value"}}'
```

Note: Replace `YOUR_GITHUB_TOKEN`, `OWNER`, and `REPO` with your actual values.

Task:

1. Create the workflow with `repository_dispatch` trigger
2. Generate a GitHub Personal Access Token with `repo` scope
3. Use the curl command to trigger the workflow externally
4. Verify the workflow runs and displays the payload data correctly

Submission Guidelines

How to Submit:

1. **Ensure all workflows are committed and pushed:**

```
git add .  
git commit -m "Complete GitHub Actions Lecture 1 Assignment"  
git push origin main
```

2. Verify all workflows:

- Navigate to the "Actions" tab in your GitHub repository
- Ensure workflows have run successfully (or check for any errors)
- Review the workflow run logs

3. Submit your repository link:

- Send the repository URL to your mentor
- Format: `https://github.com/YOUR_USERNAME/github-actions-lecture1-assignment`

Submission Checklist:

- ☒ Repository named correctly: `github-actions-lecture1-assignment`
 - ☒ All 9 workflow files created in `.github/workflows/`
 - ☒ All workflows tested and verified to work
 - ☒ README updated with any notes or observations (optional)
 - ☒ Repository link sent to mentor
-

Learning Objectives

By completing this assignment, you will understand:

- ☒ Basic workflow syntax and structure
 - ☒ Job dependencies and execution order
 - ☒ Docker integration with GitHub Actions
 - ☒ Environment variable scoping and precedence
 - ☒ Event filtering (branches, paths)
 - ☒ Manual and external workflow triggers
 - ☒ Using GitHub Actions marketplace actions
 - ☒ Working with secrets and variables
-

Additional Resources

- [GitHub Actions Documentation](#)
- [Workflow Syntax Reference](#)
- [GitHub Actions Marketplace](#)

Good luck! 🚀